

Formation Flutter Professionnelle

En programmation (et particulièrement dans Flutter avec les requêtes réseau vers Spring Boot), la gestion du temps est cruciale. Lorsqu'on demande quelque chose au serveur, cela prend quelques millisecondes ou secondes.

Pour éviter que l'application ne se fige pendant ce temps d'attente, Dart utilise trois mots-clés fondamentaux : **Future**, **async** et **await**.

Voici une explication simple, imagée et technique.

L'analogie du restaurant

Imagine que tu commandes un plat au restaurant :

1. **Sans asynchronisme (Synchrone)** : Tu commandes, et tu restes planté devant le cuisinier, bloqué, sans pouvoir bouger ni parler à personne jusqu'à ce que ton plat soit prêt. Personne d'autre ne peut commander. (C'est ce qui ferait **figer** l'écran de ton téléphone).
2. **Avec asynchronisme (async/await/Future)** : Tu commandes, on te donne un **ticket (un Future)**. Tu vas t'asseoir, tu peux discuter, utiliser ton téléphone (l'interface reste fluide). Quand le plat est prêt, on t'appelle et tu récupères ton repas.

1. Future (La Promesse)

Un **Future** représente une valeur qui n'est **pas encore disponible**, mais qui le sera **plus tard** (dans le futur).

- C'est une **promesse** que l'opération va se terminer (soit par un succès avec la donnée, soit par une erreur).
- *Exemple* : `Future<void> ajouterEtudiant(...)` signifie : *"Je promets d'exécuter cette action d'ajout, et je te préviendrai quand ce sera fini."*

2. async (Le signal)

Le mot-clé **async** (pour *asynchronous*) se place à côté d'une fonction.

- Il prévient Dart : *"Attention, cette fonction contient des tâches qui prennent du temps (comme une requête HTTP)."*
- Il autorise l'utilisation du mot-clé **await** à l'intérieur de cette fonction.

3. await (L'attente intelligente)

Le mot-clé **await** (qui signifie *"attendre"*) s'utilise **uniquement** à l'intérieur d'une fonction **async**.

- Il dit à Dart : *"Attends que cette ligne de code (par exemple, la requête Dio vers Spring Boot) soit complètement terminée avant de passer à la ligne suivante."*

- **Le grand pouvoir de `await`** : Il met en pause *uniquement* cette fonction précise. Le reste de l'application (l'interface utilisateur, les animations, les boutons) continue de tourner normalement. L'utilisateur ne voit aucun blocage.

Exemple concret dans notre code :

Dart

```
// 1. On déclare la fonction avec 'async' car elle va faire une requête
réseau
Future<void> soumettreFormulaire() async {

    print('1. Début de la soumission');

    // 2. 'await' met la fonction en pause le temps que Dio discute avec
    Spring Boot.
    // L'application ne fige pas, l'utilisateur peut toujours scroller.
    try {
        await _etudiantService.ajouterEtudiant(nouvelEtudiant);
        print('2. Succès ! L\'étudiant est enregistré.');
```

En résumé :

- **Future** = Le ticket qui annonce *"Le résultat arrivera plus tard"*.
- **async** = Le panneau qui indique *"Cette fonction gère du temps d'attente"*.
- **await** = L'ordre de dire *"Attends sagement que le résultat arrive ici sans bloquer l'écran"*.

L'Ajout d'un Étudiant en utilisant la pile standard de production : **Modèle + Service Dio + Page de Formulaire.**

Chaque fichier contient des commentaires détaillés pour vous expliquer le rôle de chaque ligne.

1. Le Modèle : `lib/features/etudiants/data/models/etudiant.dart`

- **Rôle** : Représente l'objet métier et gère la conversion au format JSON attendu par votre API Spring Boot.

Dart

```
class Etudiant {
    final String matricule;
```

```

final String nom;
final String prenom;
final String email;
final String filiere;

const Etudiant({
  required this.matricule,
  required this.nom,
  required this.prenom,
  required this.email,
  required this.filiere,
});

/// Convertit l'objet Dart en Map<String, dynamic> (JSON)
/// pour l'envoyer dans le corps (body) de la requête POST vers Spring
Boot.
Map<String, dynamic> toJson() {
  return {
    'matricule': matricule,
    'nom': nom,
    'prenom': prenom,
    'email': email,
    'filiere': filiere,
  };
}
}

```

2. Le Service Dio :

lib/features/etudiants/data/services/etudiant_service.dart

- **Rôle :** Centralise la communication réseau sécurisée avec le backend. Il configure l'URL, les délais d'attente et gère les erreurs HTTP.

Dart

```

import 'package:dio/dio.dart';
import '../models/etudiant.dart';

class EtudiantService {
  // Initialisation et configuration globale de Dio
  final Dio _dio = Dio(
    BaseOptions(
      // 10.0.2.2 est l'adresse IP spéciale de l'émulateur Android pour
      // atteindre le localhost de votre PC (Spring Boot)
      baseUrl: 'http://10.0.2.2:8080/api/v1',
      // Sécurité production : Timeout pour éviter que l'app ne bloque si
      // le serveur est absent
      connectTimeout: const Duration(seconds: 10),
      receiveTimeout: const Duration(seconds: 10),
      headers: {'Content-Type': 'application/json'},
    ),
  );

  /// Méthode d'envoi d'un nouvel étudiant vers l'API Spring Boot
  Future<void> ajouterEtudiant(Etudiant etudiant) async {
    try {
      // Exécution de la requête POST vers l'endpoint '/etudiants'
      // Dio convertit automatiquement l'objet JSON de .toJson() en texte
      brut
      await _dio.post('/etudiants', data: etudiant.toJson());
    }
  }
}

```

```

    } on DioException catch (e) {
      // Gestion professionnelle des erreurs réseau (Codes HTTP, Timeouts,
etc.)
      throw _gererErreurDio(e);
    }
  }

  /// Fonction utilitaire pour traduire les erreurs techniques en messages
  clairs
  String _gererErreurDio(DioException e) {
    switch (e.type) {
      case DioExceptionType.connectionTimeout:
      case DioExceptionType.receiveTimeout:
        return 'Le serveur met trop de temps à répondre (Timeout).';
      case DioExceptionType.badResponse:
        final statusCode = e.response?.statusCode;
        if (statusCode == 409) return 'Erreur : Ce matricule ou email
existe déjà.';
        if (statusCode == 400) return 'Requête invalide, vérifiez les
données.';
        return 'Erreur serveur (Code : $statusCode).';
      default:
        return 'Impossible de joindre le serveur. Vérifiez votre
connexion.';
    }
  }
}

```

3. La Page du Formulaire :

lib/features/etudiants/presentation/views/etudiant_ajout_page.dart

- **Rôle :** Fournit l'interface graphique (UI) avec les champs de saisie, les validations et le déclenchement de l'appel au service.

Dart

```

import 'package:flutter/material.dart';
import '../data/models/etudiant.dart';
import '../data/services/etudiant_service.dart';

class EtudiantAjoutPage extends StatefulWidget {
  const EtudiantAjoutPage({super.key});

  @override
  State<EtudiantAjoutPage> createState() => _EtudiantAjoutPageState();
}

class _EtudiantAjoutPageState extends State<EtudiantAjoutPage> {
  // Clé globale pour valider le formulaire
  final _formKey = GlobalKey<FormState>();

  // Contrôleurs pour écouter et récupérer le texte saisi dans chaque champ
  final _matriculeController = TextEditingController();
  final _nomController = TextEditingController();
  final _prenomController = TextEditingController();
  final _emailController = TextEditingController();
  final _filierController = TextEditingController();

  // Instance du service Dio
  final EtudiantService _etudiantService = EtudiantService();

```



```
// État local pour afficher un indicateur de chargement (Spinner) pendant
la requête
bool _isLoading = false;

@Override
void dispose() {
    // Nettoyage indispensable des contrôleurs pour éviter les fuites de
mémoire (Memory Leaks)
    _matriculeController.dispose();
    _nomController.dispose();
    _prenomController.dispose();
    _emailController.dispose();
    _filierController.dispose();
    super.dispose();
}

/// Logique exécutée au clic sur le bouton de soumission
Future<void> _soumettre() async {
    // 1. Valide si tous les champs respectent les règles de validation
    if (!_formKey.currentState!.validate()) return;

    // 2. Active le chargement visuel
    setState(() => _isLoading = true);

    try {
        // 3. Crée l'objet modèle avec les données récupérées des contrôleurs
        final nouvelEtudiant = Etudiant(
            matricule: _matriculeController.text.trim(),
            nom: _nomController.text.trim(),
            prenom: _prenomController.text.trim(),
            email: _emailController.text.trim(),
            filiere: _filierController.text.trim(),
        );

        // 4. Envoie l'étudiant via le service Dio vers Spring Boot
        await _etudiantService.ajouterEtudiant(nouvelEtudiant);

        // 5. Vérifie si le widget est toujours visible avant d'interagir
avec le contexte
        if (mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                    content: Text('Étudiant enregistré avec succès !'),
                    backgroundColor: Colors.green,
                ),
            );
            Navigator.of(context).pop(); // Ferme la page et retourne en
arrière
        }
    } catch (e) {
        // Gestion de l'affichage de l'erreur renvoyée par le service
        if (mounted) {
            ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                    content: Text(e.toString().replaceAll('Exception: ', '')),
                    backgroundColor: Colors.red,
                ),
            );
        }
    } finally {
        // 6. Désactive le chargement dans tous les cas (succès ou échec)
    }
}
```



```
        if (mounted) {
          setState(() => _isLoading = false);
        }
      }
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Ajouter un Étudiant'),
      ),
      body: Padding(
        padding: const EdgeInsets.all(16.0),
        child: Form(
          key: _formKey, // Associe la clé de validation au formulaire
          child: ListView(
            children: [
              TextFormField(
                controller: _matriculeController,
                decoration: const InputDecoration(labelText: 'Matricule'),
                validator: (value) => value == null || value.isEmpty ? 'Ce
champ est requis' : null,
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _prenomController,
                decoration: const InputDecoration(labelText: 'Prénom'),
                validator: (value) => value == null || value.isEmpty ? 'Ce
champ est requis' : null,
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _nomController,
                decoration: const InputDecoration(labelText: 'Nom'),
                validator: (value) => value == null || value.isEmpty ? 'Ce
champ est requis' : null,
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _emailController,
                decoration: const InputDecoration(labelText: 'Email'),
                validator: (value) => value == null || value.isEmpty ? 'Ce
champ est requis' : null,
              ),
              const SizedBox(height: 12),
              TextFormField(
                controller: _filiereController,
                decoration: const InputDecoration(labelText: 'Filière'),
                validator: (value) => value == null || value.isEmpty ? 'Ce
champ est requis' : null,
              ),
              const SizedBox(height: 24),
              ElevatedButton(
                // Désactive le bouton si _isLoading est vrai pour éviter
                les double-clics
                onPressed: _isLoading ? null : _soumettre,
                child: _isLoading
                  ? const SizedBox(
                      height: 20,
                      width: 20,
```



```
        child: CircularProgressIndicator(strokeWidth: 2,  
color: Colors.white),  
      ),  
      : const Text('Enregistrer dans la base'),  
    ),  
  ],  
),  
),  
),  
),  
);  
}  
}
```